

Simple Maven Project

Introduction to Maven

Apache Maven is an open-source build automation and project management tool mainly used for Java projects. In DevOps, Maven helps developers automate tasks such as compiling code, testing, packaging, and managing project dependencies.

Maven follows a standard project structure and uses an XML file called pom.xml (Project Object Model) to manage project configuration.

Brief Explanation of Maven

Maven simplifies the software development process by automating the build lifecycle. It is widely used in DevOps for Continuous Integration (CI) and Continuous Deployment (CD).

Key Features of Maven

1. **Build Automation**
 - Automatically compiles source code and creates application packages like JAR or WAR files.
2. **Dependency Management**
 - Downloads required libraries from online repositories automatically.
3. **Standard Project Structure**
 - Provides a fixed directory structure, making projects easier to manage.
4. **Easy Integration**
 - Works well with DevOps tools such as:
 - Jenkins
 - Git
 - Docker
5. **Testing Support**
 - Supports automated testing using frameworks like JUnit.

Working of Maven in DevOps

1. Developer writes code.
2. Code is pushed to Git repository.
3. Jenkins triggers Maven build.
4. Maven compiles code and runs tests.
5. Maven packages the application.
6. Application is deployed to the server.

Advantages of Maven

- Simple and easy to use
- Reduces manual work
- Automatic dependency management
- Faster project setup

- Supports CI/CD pipelines

Conclusion

Maven is an important DevOps tool used for automating software builds and managing dependencies. It improves productivity, ensures consistency, and supports continuous integration and deployment in modern software development.

Step1:

To create a basic Maven Java project, run the following command:

```
mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenApp -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```



```

vaishnav@vaishnavi-virtualbox: ~/devops2
$ mvn archetype:generate
[INFO] --- maven-archetype-plugin/3.4.1:generate (default-cli) --- generate-sources @ standalone
[INFO]
[INFO] --- maven-archetype-plugin/3.4.1:generate (default-cli) @ standalone ---
[INFO] Generating project in Batch mode
[INFO] Downloading from central: https://repo.maven.apache.org/maven2/archetype-catalog.xml
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/archetype-catalog.xml (16 kB at 2.8 MB/s)
[INFO] No archetype defined. Using maven-archetype-quickstart (org.apache.maven.archetypes:maven-archetype-quickstart:1.1.0)
[INFO]
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-quickstart:1.1.0
[INFO]
[INFO] Parameter: basedir, Value: /home/vaishnavi/devops2
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: MyMavenApp
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/vaishnavi/devops2/MyMavenApp
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 7.847 s
[INFO] Finished at: 2020-01-24T11:11:06+05:30
[INFO]
vaishnavi@vaishnavi-virtualbox: ~/devops2

```

The command can be broken down as:

`DgroupId=com.example` – project's base package (reverse-domain style).

`-DartifactId=my-maven-app` – defines the name of the project (will also be the folder name).

`-DarchetypeArtifactId=maven-archetype-quickstart` – sets the Maven template to be the basic Java application.

`-DinteractiveMode=false` – deactivates interactive mode and uses the given values automatically.

Step 2:

Use `$cd` command to go into the directory.

Execute `$tree` command to view the structure of the project

```
vaishnavi@vaishnavi-virtualbox:~/devops21$ cd MyMavenApp1
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenApp1$ tree
```

```

.
├── pom.xml
└── src
    ├── main
    │   ├── java
    │   │   ├── com
    │   │   │   └── example
    │   │       └── App.java
    └── test
        ├── java
        │   ├── com
        │   │   └── example
        │       └── AppTest.java

```

9 directories, 3 files

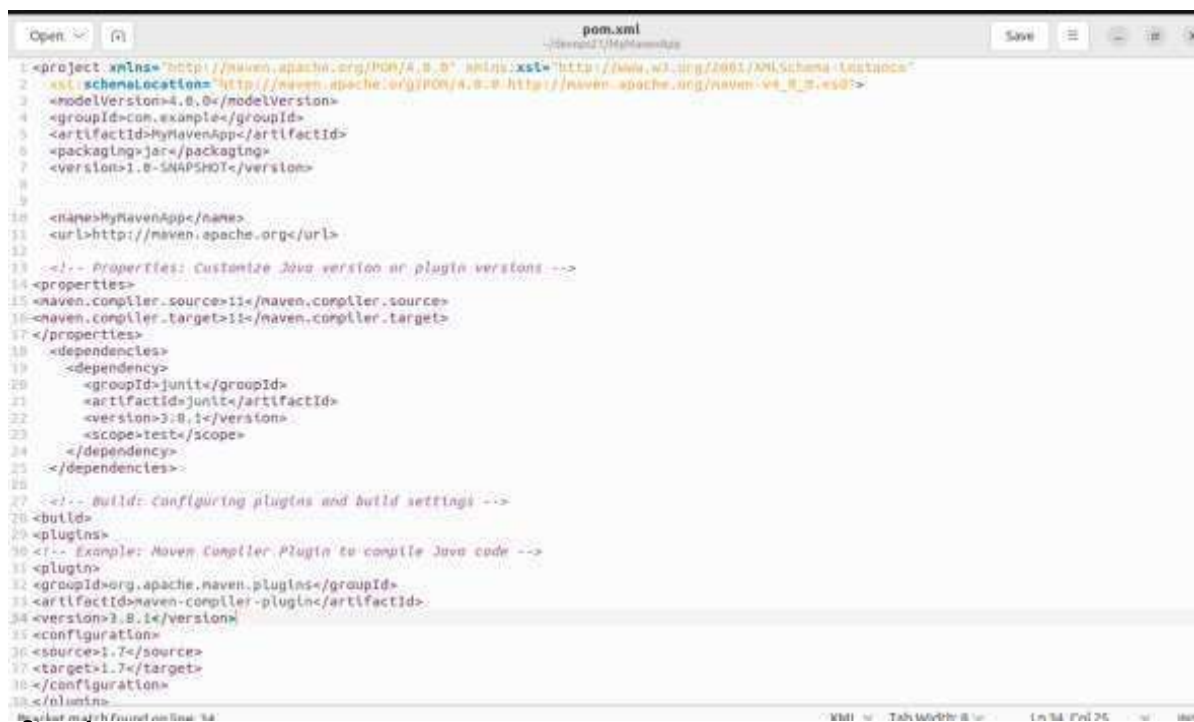
```
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenApp1$
```

Steo 3:

Edit the pom.xml file

Add the required dependencies and plugins required from central maven respository

For given project, we will download unit testing dependency from Jenkins.



```

pom.xml
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
4   <groupId>com.example</groupId>
5   <artifactId>MyMavenApp</artifactId>
6   <packaging>jar</packaging>
7   <version>1.0-SNAPSHOT</version>
8
9   <name>MyMavenApp</name>
10  <url>http://maven.apache.org</url>
11
12  <!-- Properties: Customize Java version or plugin versions -->
13  <properties>
14    <maven.compiler.source>11</maven.compiler.source>
15    <maven.compiler.target>11</maven.compiler.target>
16  </properties>
17
18  <dependencies>
19    <dependency>
20      <groupId>junit</groupId>
21      <artifactId>junit</artifactId>
22      <version>3.8.1</version>
23      <scope>test</scope>
24    </dependency>
25  </dependencies>
26
27  <!-- Build: Configuring plugins and build settings -->
28  <build>
29    <plugins>
30      <!-- Example: Maven Compiler Plugin to compile Java code -->
31      <plugin>
32        <groupId>org.apache.maven.plugins</groupId>
33        <artifactId>maven-compiler-plugin</artifactId>
34        <version>3.8.1</version>
35        <configuration>
36          <source>1.7</source>
37          <target>1.7</target>
38        </configuration>
39      </plugin>
40    </plugins>
41  </build>
42</project>

```

Step 4:

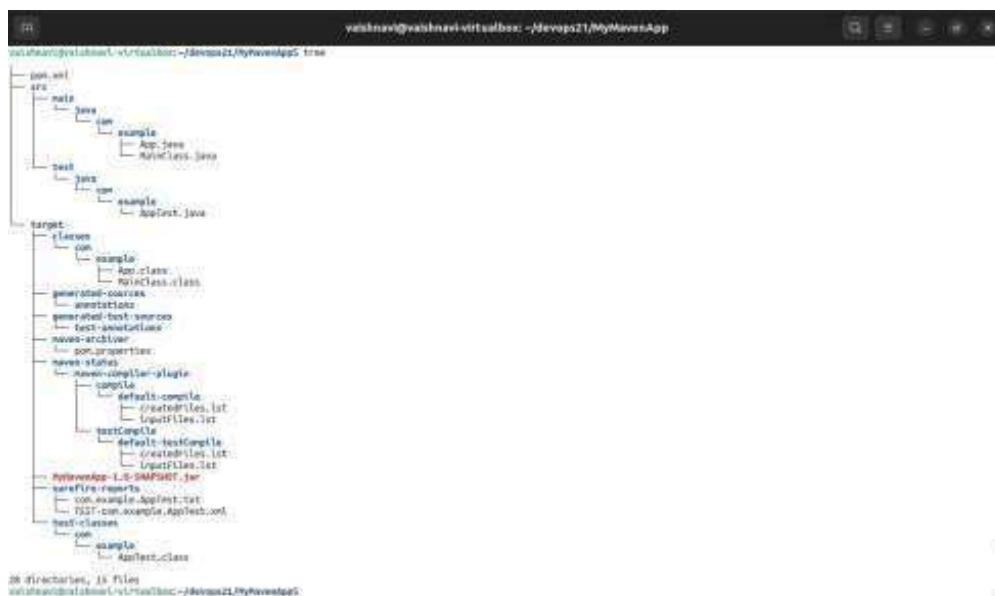
Execute the command \$mvn compile

If the command shoes output as “build success”, it means that the project has compiled successfully

```
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenApp$ mvn compile
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:MyMavenApp >-----
[INFO] Building MyMavenApp 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ MyMavenApp ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is plat
form dependent!
[INFO] skip non existing resourceDirectory /home/vaishnavi/devops21/MyMavenApp/src/main/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:compile (default-compile) @ MyMavenApp ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 0.713 s
[INFO] Finished at: 2026-03-24T21:20:16+05:30
[INFO]
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenApp$
```

Step 5:

Execute \$tree command again to view the complete structure after adding plugins and dependencies to the pom.xml file



```
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenApp$ tree
.
├── pom.xml
├── src
│   ├── main
│   │   ├── java
│   │   │   └── com
│   │   │       └── example
│   │   │           ├── App.java
│   │   │           └── MainClass.java
│   └── test
│       ├── java
│       │   └── com
│       │       └── example
│       │           └── AppTest.java
└── target
    ├── classes
    │   └── com
    │       └── example
    │           ├── App.class
    │           └── MainClass.class
    ├── generated-sources
    │   └── annotations
    │       └── test-annotations
    ├── generated-test-sources
    │   └── test-annotations
    ├── maven-archiver
    │   └── pom.properties
    ├── maven-status
    │   └── maven-compiler-plugin
    │       └── compile
    │           ├── default-compile
    │           │   ├── createdFiles.lst
    │           │   └── inputFiles.lst
    │           └── testCompile
    │               ├── default-testCompile
    │               │   ├── createdFiles.lst
    │               │   └── inputFiles.lst
    └── MyMavenApp-1.0-SNAPSHOT.jar
        ├── surefire-reports
        │   ├── com.example.AppTest.txt
        │   └── TEST-com.example.AppTest.txt
        ├── test-classes
        │   └── com
        │       └── example
        │           └── AppTest.class
        └── example
            └── AppTest.class

28 directories, 15 files
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenApp$
```

Step 6:

Execute \$mvn test to run unit tests

```

valsham@valsham:~/virtualbox/~/devops21/MyMavenApp$ mvn test
[INFO] Scanning for projects...
[INFO]
[INFO] -----[ com.example.MyMavenApp ]-----
[INFO] Building MyMavenApp 1.0-SNAPSHOT
[INFO]
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ MyMavenApp ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/valsham/devops21/MyMavenApp/src/main/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:compile (default-compile) @ MyMavenApp ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- maven-resources-plugin:2.6:testResources (default-testResources) @ MyMavenApp ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/valsham/devops21/MyMavenApp/src/test/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:testCompile (default-testCompile) @ MyMavenApp ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- maven-surefire-plugin:2.22.1:test (default-test) @ MyMavenApp ---
[INFO]
[INFO] -----[ T E S T S ]-----
[INFO]
[INFO] Running com.example.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.003 s - in com.example.AppTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----[ BUILD SUCCESS ]-----
[INFO]
[INFO] Total time: 1.303 s
[INFO] Finished at: 2020-03-24T21:27:45+05:30
[INFO]
valsham@valsham:~/virtualbox/~/devops21/MyMavenApp$
valsham@valsham:~/virtualbox/~/devops21/MyMavenApp$

```

Step 7:

Run mvn package command to generate a JAR application bundle which contains our basic Maven Hello World application

```

valsham@valsham:~/virtualbox/~/devops21/MyMavenApp$ mvn package
[INFO] Scanning for projects...
[INFO]
[INFO] -----[ com.example.MyMavenApp ]-----
[INFO] Building MyMavenApp 1.0-SNAPSHOT
[INFO]
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ MyMavenApp ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/valsham/devops21/MyMavenApp/src/main/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:compile (default-compile) @ MyMavenApp ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- maven-resources-plugin:2.6:testResources (default-testResources) @ MyMavenApp ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/valsham/devops21/MyMavenApp/src/test/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:testCompile (default-testCompile) @ MyMavenApp ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- maven-surefire-plugin:2.22.1:test (default-test) @ MyMavenApp ---
[INFO]
[INFO] -----[ T E S T S ]-----
[INFO]
[INFO] Running com.example.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.003 s - in com.example.AppTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- maven-jar-plugin:3.1.0:jar (default-jar) @ MyMavenApp ---
[INFO]
[INFO] -----[ BUILD SUCCESS ]-----
[INFO]
[INFO] Total time: 1.468 s
[INFO] Finished at: 2020-03-24T21:28:09+05:30
[INFO]
valsham@valsham:~/virtualbox/~/devops21/MyMavenApp$
valsham@valsham:~/virtualbox/~/devops21/MyMavenApp$

```

Step 8:

Run the following command to run the JAR file: `java -jar target/MyMavenApp-1.0-SNAPSHOT.jar`

```
[INFO] -----
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenApp$ java -jar target/MyMavenApp-1.0-SNAPSHOT.jar
HelloClass, happy to see you all
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenApp$ █
```

Pushing The Project into Git:

Execute the commands:

```
$ git config --global user.name "vaishnavim1121"
```

```
$ git config --global user.email "vaishnavim1121@gmail.com"
```

```
$git init
```

```
$git add .
```

```
$git commit -m "initial commit"
```

```
$ git remote add origin https://github.com/vaishnavim1121/MyMavenApp.git
```

```
$ git remote set-url origin https://*PAT*@github.com/vaishnavim1121/MyMavenApp
```

```
$ git push -u origin master
```

```

└── AppTest.class
    └── AppTest.class

30 directories, 19 files
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenToGradle$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/vaishnavi/devops21/MyMavenToGradle/.git/
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenToGradle$ git add .
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenToGradle$ git commit -m "naveel to gradle"
[master (root-commit) 255ca20] naveel to gradle
 27 files changed, 473 insertions(+)
 create mode 100644 .gradle/4.4.1/fileChanges/last-build.bin
 create mode 100644 .gradle/4.4.1/fileHashes/fileHashes.bin
 create mode 100644 .gradle/4.4.1/fileHashes/fileHashes.lock
 create mode 100644 .gradle/4.4.1/taskHistory/taskHistory.bin
 create mode 100644 .gradle/4.4.1/taskHistory/taskHistory.lock
 create mode 100644 .gradle/buildOutputCleanup/buildOutputCleanup.lock
 create mode 100644 .gradle/buildOutputCleanup/cache.properties
 create mode 100644 .gradle/buildOutputCleanup/outputFiles.bin
 create mode 100644 build.gradle
 create mode 100644 gradle/wrapper/gradle-wrapper.jar
 create mode 100644 gradle/wrapper/gradle-wrapper.properties
 create mode 100755 gradlew
 create mode 100644 gradlew.bat
 create mode 100644 pom.xml
 create mode 100644 settings.gradle
 create mode 100644 src/main/java/com/example/App.java
 create mode 100644 src/test/java/com/example/AppTest.java
 create mode 100644 target/MyMavenApp-1.0-SNAPSHOT.jar
 create mode 100644 target/classes/com/example/App.class
 create mode 100644 target/maven-archiver/pom.properties
 create mode 100644 target/maven-status/maven-compiler-plugin/compile/default-compile/createdFiles.lst
 create mode 100644 target/maven-status/maven-compiler-plugin/compile/default-compile/inputFiles.lst
 create mode 100644 target/maven-status/maven-compiler-plugin/testCompile/default-testCompile/createdFiles.lst
 create mode 100644 target/maven-status/maven-compiler-plugin/testCompile/default-testCompile/inputFiles.lst
 create mode 100644 target/surefire-reports/TEST-com.example.AppTest.xml
 create mode 100644 target/surefire-reports/com.example.AppTest.txt
 create mode 100644 target/test-classes/com/example/AppTest.class
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenToGradle$ git remote add origin https://github.com/vaishnavim1121/MyMavenToGradle.git
git: 'remote' is not a git command. See 'git --help'.
```

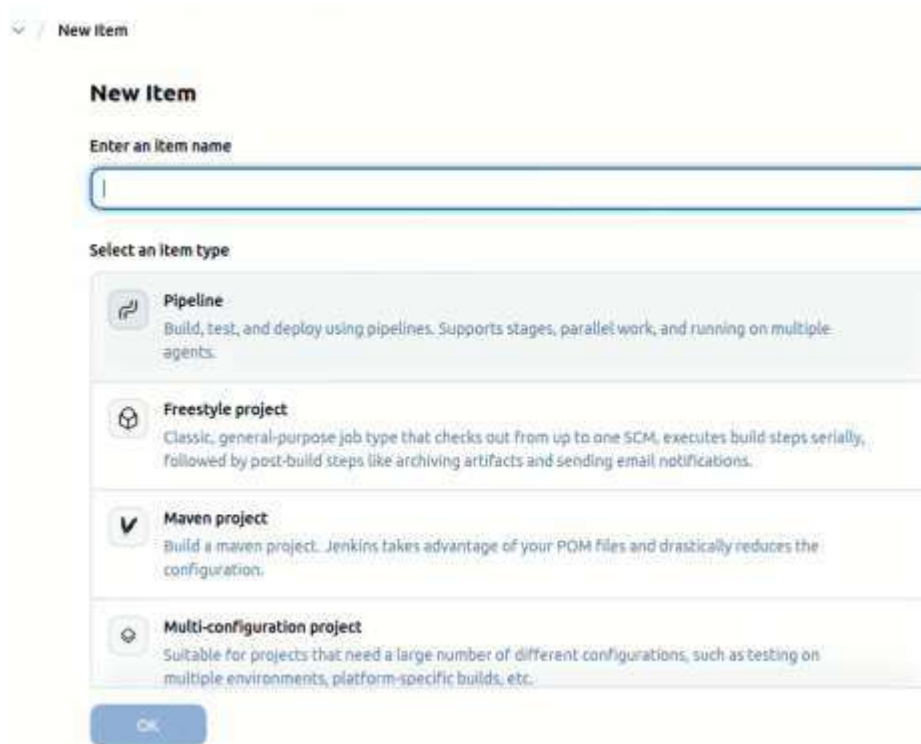

Second Phase

Depicting Jenkins Pipeline

Follow the given steps to deploy the project on Jenkins.

create a Jenkins pipeline job that will use the Jenkinsfile from your project repository

go to the Jenkins home page and click on “New Item” to create a job, enter the item name select Pipeline as the project type.



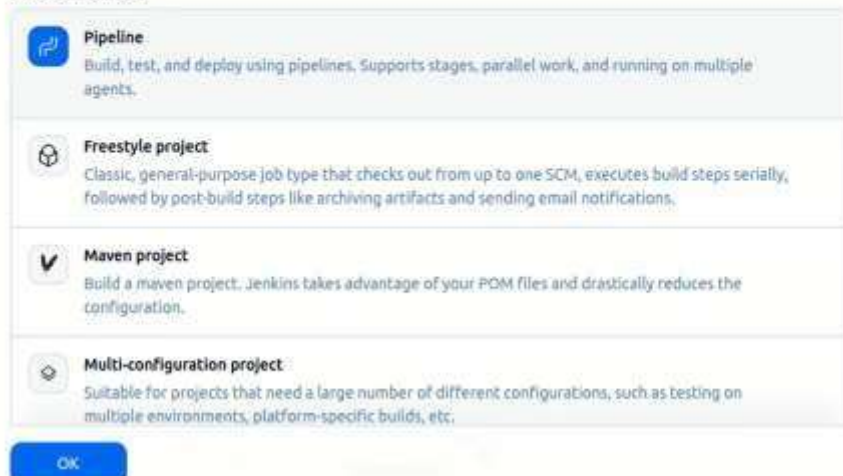
The screenshot shows the Jenkins 'New Item' configuration page. At the top, there is a breadcrumb 'New Item'. Below it, the title 'New Item' is displayed. A text input field labeled 'Enter an item name' is empty. Under the 'Select an item type' section, four options are listed: 'Pipeline' (selected with a checkmark), 'Freestyle project', 'Maven project', and 'Multi-configuration project'. Each option has a brief description. At the bottom, there is a blue 'OK' button.

New Item

Enter an item name

MyMavenApp

Select an item type



This screenshot shows the same Jenkins 'New Item' form as the previous one, but with the item name 'MyMavenApp' entered in the text field. The 'Pipeline' option remains selected under the 'Select an item type' section. The 'OK' button is still at the bottom.

This option allows you to build, test, and deploy applications using a Jenkinsfile. After selecting the pipeline type, click “OK” to proceed to the job configuration page

Definition

Pipeline script from SCM

SCM

Git

Repositories

Repository URL

https://github.com/vaishnavim1121/MyMavenApp.git

Please enter Git repository.

Credentials

github credentials

+ Add

Advanced

Save Apply

Now we need to add a jenkinsfile into our project which will consist of the operations that will occur when building the project in Jenkins

It includes step wise instructions guiding at each stage with right commands

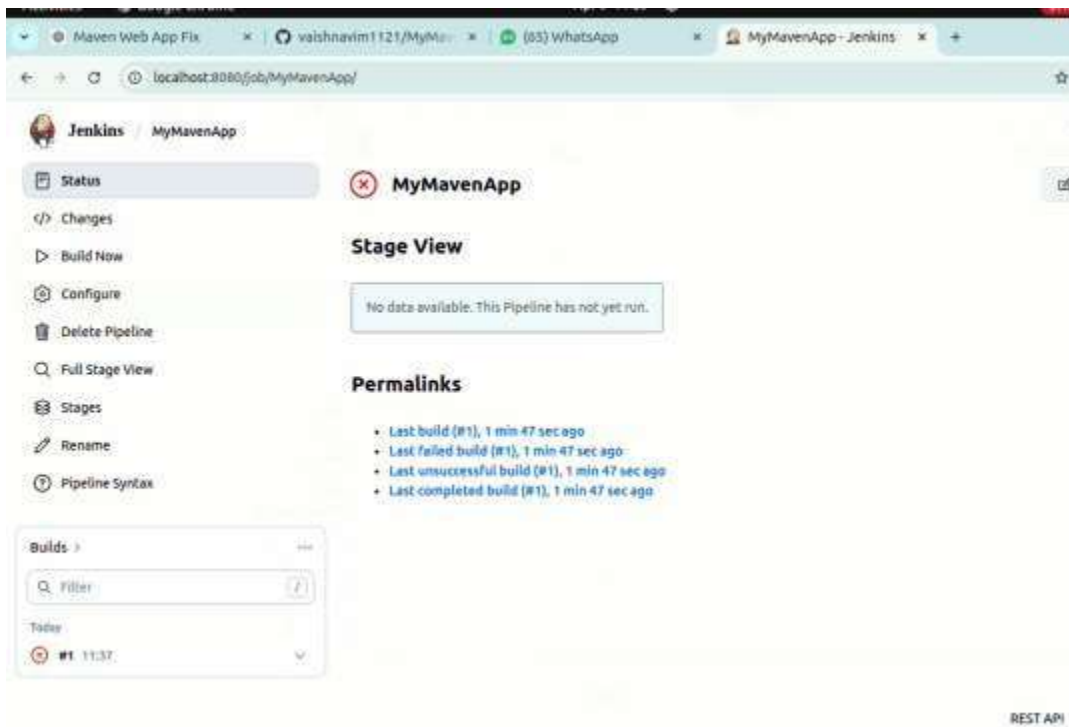
```

1 pipeline {
2   agent any // Use any available agent
3
4   tools {
5     maven 'Haven' // Ensure this matches the name configured in Jenkins
6   }
7   stages {
8     stage('Checkout') {
9       steps {
10        git branch: 'master', url: 'https://github.com/vaishnavim1121/MyMavenApp.git'
11      }
12    }
13    stage('Build') {
14      steps {
15        sh 'mvn clean package' // Run Maven build
16      }
17    }
18    stage('Test') {
19      steps {
20        sh 'mvn test' // Run unit tests
21      }
22    }
23
24    stage('Run Application') {
25      steps {
26        // Start the JAR application
27        sh 'java -jar target/MyMavenApp-1.0-SNAPSHOT.jar'
28      }
29    }
30  }
31 }
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

Plain Text Tab Width: 8

Now try building the project with Jenkins



The error occurs because there is no Jenkinsfile in the github repository. Thus we need to push the content to our project repository in order to run the project successfully.

Push the project again to your git account

Now try building the project in Jenkins by clicking on build-now option.

MyMavenApp - Stage View

	Declarative: Checkout SCM	Declarative: Tool Install	Checkout	Build	Test	Run Application	Declarative: Post Actions
Average stage times:	2s	550ms	1s	51s	410ms	306ms	306ms
#1 Apr 09 11:47 No Changes	2s	550ms	1s	51s Failed	410ms Failed	306ms Failed	306ms
#1 Apr 09 11:37 No Changes							

Check the logs in order to rectify the error.

The error is due to version being outdated,



In order to correct it, edit the version that is mentioned in the pom.xml file.
 Change the version from 1.7 to 11 in both source and target tabs.
 After committing the changes, build again in the Jenkins

MyMavenApp - Stage View

